

Golden Thread — Obsidian & Daily Workflow Guide

How to use Golden Thread day-to-day, combining Obsidian as your reading/navigation layer with Claude Code skills as your writing layer.

The Division of Labor

YOU USE...	FOR...
Obsidian	Reading, browsing, graph navigation, search, reviewing what exists
Claude Code skills	Writing, updating, promoting, maintaining the vault

The vault is plain markdown files. Obsidian is the human GUI. Claude Code is the AI's write interface. You can edit vault files directly in Obsidian — but the skills handle the ceremony (frontmatter, cross-links, log entries, index updates) so you don't have to.

How the Vault Looks in Obsidian

THE FILE TREE

```
GoldenThreadVault/
  CLAUDE.md          ← AI instructions (don't edit manually)
  index.md           ← Knowledge page catalog – your navigation hub
  log.md             ← audit trail of every operation
  review-queue.md    ← items flagged for your review

  Sources/           ← raw ingested docs – never edit these
  Knowledge/         ← cross-project wiki pages
    _template.md     ← page format reference (don't use as a page)

  global-memory/     ← facts loaded in every Claude session
    MEMORY.md        ← one-line index (this is what Claude reads)

  Projects/
    README.md        ← master project list (open this first)
    CONVENTIONS.md   ← how projects are structured
    PROTOCOL.md      ← cross-project process rules
    INFRASTRUCTURE.md ← your server fleet (defined once, linked everywhere)

  shel/              ← a project
    README.md        ← status board – your daily check-in file
    idea.md          ← origin story (never changes)
    research.md       ← findings (append-only)
    decisions.md      ← ADRs (append-only)
    design.md         ← current architecture
    runbook.md        ← operational commands
    harvester/        ← sub-project (same structure)
    memory/
      MEMORY.md      ← session memory index
```

KEY OBSIDIAN VIEWS

- **Graph view** — shows how Knowledge pages link to each other. Clusters reveal related areas. Isolated nodes are orphans (`gt-lint` will catch them).
 - **Backlinks panel** — open a page and see everything that links to it. Use this to understand impact before changing a Knowledge page.
 - **Search** — vault-wide full text. Faster than `gt-query` for exact strings you know exist.
 - **Tag pane** — project tags from `README.md` frontmatter. Useful for filtering by domain.
 - **Dataview plugin** (if installed) — `Projects/README.md` contains Dataview queries that auto-generate project status tables from frontmatter.
-

Daily Workflow — Step by Step

MORNING: START A SESSION

1. Check your project status in Obsidian

Open `Projects/<slug>/README.md`. The status board shows sub-projects, current phase, and next action. This is your daily brief — read it before opening Claude Code.

2. Open the project in Claude Code

```
/gt:gt-open <project-slug>
```

Claude reads all project docs (idea → research → decisions → design → spec → runbook) plus the memory index, then summarizes: current stage, next action, blockers. Ask it to load specific memory files if you need session state from last time.

3. (Optional) Check the wiki

If you're about to work on something that might already be documented:

```
/gt:gt-wiki <question>
```

The wiki is checked before the web — if your team has already figured something out, it's here.

DURING THE SESSION: ACTIVE WORK

Looking something up

```
/gt:gt-query <question>
```

Reads `index.md`, follows wikilinks, falls back to grep. Use this for platform behavior, conventions, past decisions. If a result shows `status: stale`, verify it before acting on it.

Adding a source to the wiki

```
/gt:gt-wiki-ingest <url or file path>
```

Stores the source immutably, discusses what pages it should generate, then waits for your approval before writing anything.

Making a decision mid-session

Don't wait until `gt-work` for architectural decisions. Capture them immediately:

"Add an ADR: we're switching from polling to webhooks because..."

Claude will write the numbered ADR to `decisions.md`. ADRs are append-only — to reverse one, add a new ADR that supersedes it.

Writing something to the runbook

Any command, landmine, or setup step worth keeping:

"Add this to the runbook: `node setup.js` must be re-run after token expiry (~24h)"

Claude appends it to `runbook.md`. Runbook is the incubator — facts that survive several sessions get promoted to the repo's `CLAUDE.md`.

END OF SESSION: WRITE BACK

```
/gt:gt-work
```

The most important command. Claude will:

- Append dated findings to `research.md`
- Add any new ADRs to `decisions.md`
- Rewrite `design.md` if the architecture changed
- Update `memory/*.md` with session state
- Create `spec.md` if design is complete enough to hand off
- Update the `stage:` property on the README if the project phase changed

What not to skip: if you close the session without running `gt-work`, those findings exist only in Claude's context and are gone when the session ends.

WEEKLY / PERIODIC

Review daily notes

```
/gt:gt-review
```

Scans your Obsidian daily notes for tasks and ideas that were captured there but never promoted into a tracked project. Surfaces them grouped by date and lets you route each one.

Graduate a finding

```
/gt:gt-promote
```

Use when a fact has proven true across more than one project. Promotes it from project memory → Knowledge/ → global-memory/. Also handles retiring stale content and scaffolding new projects.

Check sources for changes

```
/gt:gt-wiki-refresh
```

Checks sources with `remote:` or `url:` fields for upstream changes. Supersedes changed ones with new immutable files — never edits the old source.

Lint the vault

```
/gt:gt-lint
```

Runs deterministic checks and reports findings grouped by type. You approve each fix — nothing is auto-applied. Declines are recorded in `lint-declines.md` so they don't get re-raised.

Lint runbooks

```
/gt:gt-runbook-lint
```

Scans all `runbook.md` files for procedures that have drifted into multiple runbooks. Routes duplicates to the right shared layer.

Lint the wiki

```
/gt:gt-wiki-lint
```

10 checks on the wiki: broken links, orphans, missing reciprocal links, unsourced pages, superseded sources still cited, review-due pages, index mismatches, status schema errors, declared expiries, and unlinked mentions.

Full Command Reference

SETUP (RUN ONCE)

COMMAND	WHEN TO USE
<code>/gt:gt-init</code>	First install on a new machine, or wiring an existing vault to a new project. Idempotent — safe to re-run.
<code>/gt:gt-wiki-init</code>	Set up a new wiki vault from scratch. Creates <code>Sources/</code> , <code>Knowledge/</code> , <code>index.md</code> , <code>log.md</code> , and seeds <code>CLAUDE.md</code> .

EVERY SESSION

COMMAND	WHEN TO USE
<code>/gt:gt-open</code> <code><slug></code>	Start of every session. Loads project context and asks where to pick up.
<code>/gt:gt-route</code>	Any time in the middle. Names what the session has become, where its output belongs, and whether the project, harness and model are right. Cheap — run it whenever the ground has shifted.
<code>/gt:gt-work</code>	End of every session. Writes back findings, decisions, design updates, memory state. Do not skip.

DURING WORK

COMMAND	WHEN TO USE
<code>/gt:gt-query</code> <code><question></code>	Look something up in the vault before going to the web.
<code>/gt:gt-wiki <question></code>	Query the structured wiki knowledge base.
<code>/gt:gt-wiki-ingest</code> <code><source></code>	Add a URL, file, or pasted content to the wiki. Stores immutably, generates pages with your approval.

PROMOTION AND GROWTH

COMMAND	WHEN TO USE
/gt:gt-create	Scaffold a new project. Captures your brain dump into the immutable idea.md .
/gt:gt-ingest	Import an existing project's notes, memory files, and CLAUDE.md rules into the vault.
/gt:gt-promote	Graduate a fact up the hierarchy when a second project proves it general.
/gt:gt-review	Pull uncaptured items from Obsidian daily notes into tracked projects.

MAINTENANCE (WEEKLY OR AS NEEDED)

COMMAND	WHEN TO USE
/gt:gt-lint	Audit vault structure: broken links, orphans, unlisted memory files, stale pages.
/gt:gt-runbook-lint	Find procedures duplicated across project runbooks.
/gt:gt-refresh	Check Sources/ for upstream changes and supersede outdated ones.
/gt:gt-wiki-lint	Health-check the wiki vault.
/gt:gt-wiki-refresh	Check wiki sources for upstream changes.

What to Edit in Obsidian vs. Claude Code

FILE	EDIT IN OBSIDIAN?	EDIT VIA SKILL?
Projects/<slug>/README.md	Yes — status board updates	gt-work updates stage
research.md	Read freely; add a dated note if you must	gt-work for session findings
decisions.md	Read only — append-only, don't edit past entries	gt-work / mid-session ask
design.md	Yes — light edits fine	gt-work for architecture changes
runbook.md	Yes — add commands directly	Mid-session ask or gt-work
Knowledge/*.md	Yes — corrections fine	gt-wiki-ingest for new pages
Sources/*	Never — immutable	gt-wiki-refresh for changes
global-memory/*.md	Light edits fine	gt-promote for promotion
CLAUDE.md	Avoid — AI reads this at session start	Skills seed and update this
index.md	Read only	Updated automatically by skills
log.md	Read only — append-only audit trail	Updated automatically by skills

The Promotion Ladder

When a fact graduates, it moves up:

memory/*.md

↓ gt-work

research.md

↓ gt-promote

Knowledge/

↓ gt-promote

global-memory/

this session, this project detail

settled finding, this project

true across multiple projects

needed in every session, every project

The rule: promote when a second project hits the same truth. File narrow when unsure — promotion is cheap, and a fact promoted too early becomes a rule that isn't true everywhere.

Tips for Obsidian

- Start from `Projects/README.md` not the file tree — it's the master index.
- Follow wikilinks in Knowledge pages — the graph is the navigation system, not folders.
- Use backlinks before editing a Knowledge page to see what depends on it.
- `index.md` is the Knowledge entry point — every page is one hop from there.
- `log.md` tells you what happened — if something looks wrong, check the log first.
- `review-queue.md` is written by `gt-lint` — check it after every lint run.